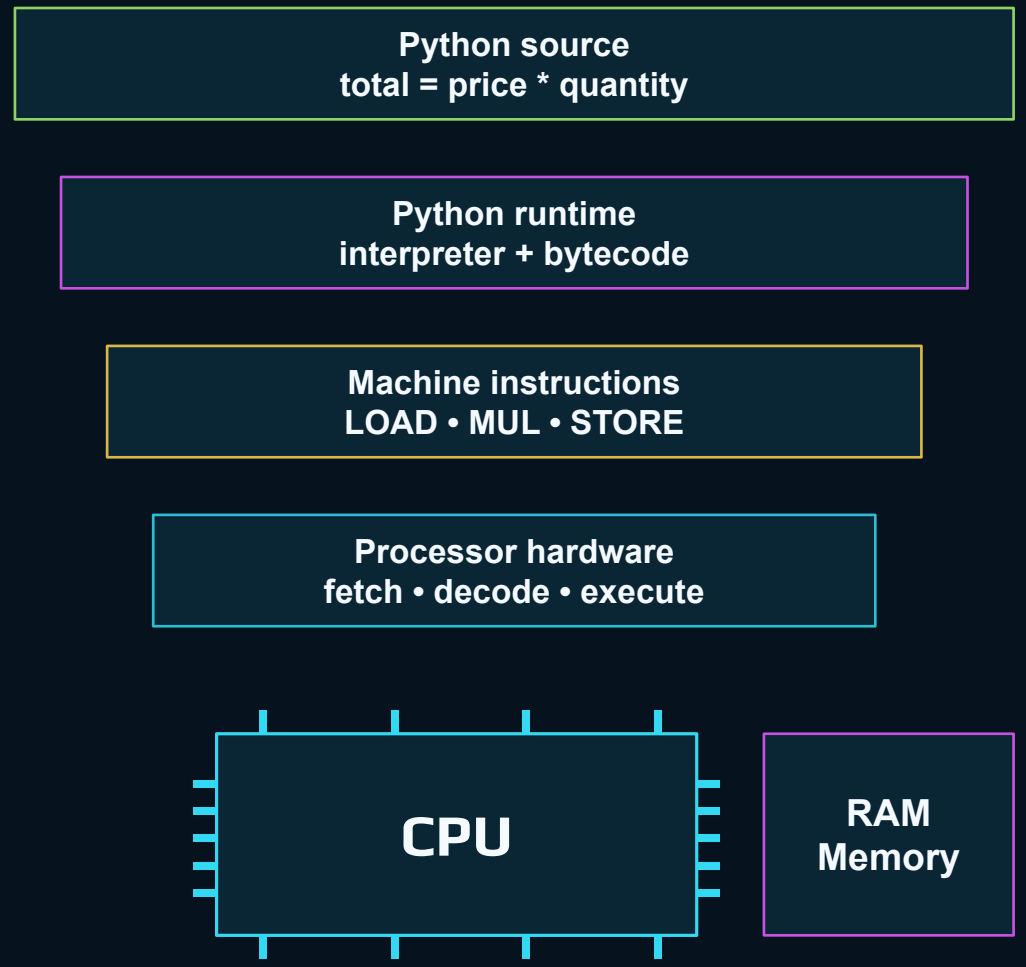
THE CPU DOES NOT SPEAK PYTHON

Processor, Memory, and Program Execution



Modern code feels human. The machine still runs tiny instructions.

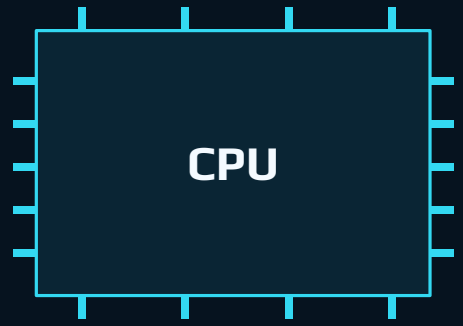
30 min instruction • 30 min guided paper-processor exercise

INSTRUCTION

Cold open: who actually executes this?

The human-friendly line is not the hardware-friendly line.

```
total = price * quantity
```




```
10110000
01101001
11100010
```

Human reads:
“multiply price by quantity”

Python translates:
source → lower-level operations

Processor executes:
machine instructions

How does the CPU understand the words price, quantity, or total?

Today’s mission

Build a mental model for what “running a program” really means.

1. **Name** — CPU, register, memory, machine code
2. **Trace** — fetch → decode → execute
3. **Compare** — source code, bytecode, machine code
4. **Simulate** — a tiny paper processor

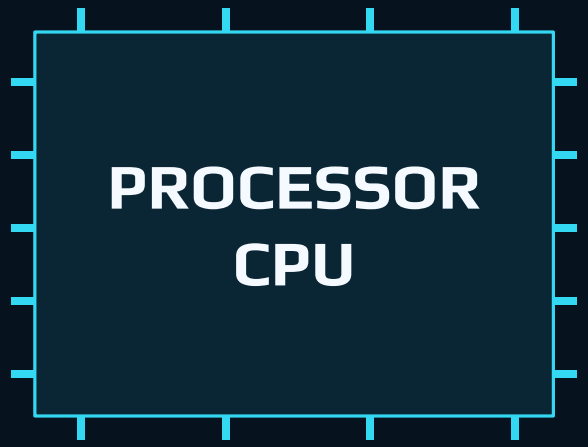
KEY IDEA

Python is a helpful layer, not the bottom layer.

The hardware still runs instructions.

By the end, students should be able to explain why high-level code is easier for humans while lower-level operations are closer to hardware.

The processor: an instruction-execution machine



LOAD 5

ADD X

COMPARE 10

JUMP LOOP

STORE RESULT

The CPU does one tiny step, then another, then another...

Billions of times per second.

A CPU is very fast, very literal, and not very imaginative.

Fun but accurate: a processor is less like a brain and more like an impossibly fast employee following exact index cards.

Machine instructions and instruction sets

Every processor family has a vocabulary of operations it understands.

Instruction set
The legal verbs for that CPU

›

Machine instruction
One specific command

›

Machine code
The encoded bits for those commands

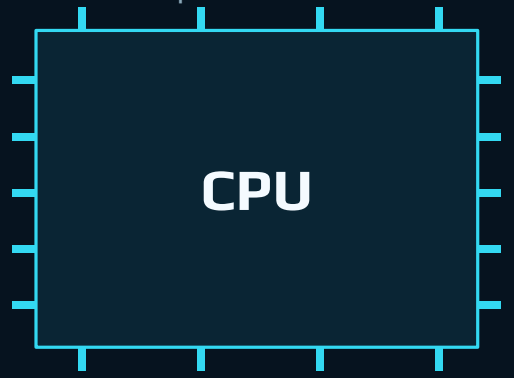
```
ADD R1, R2      // human-readable assembly style
000000 00001 00010 00011 00000 100000    // encoded machine bits
```

Different instruction sets are why software often has versions for different CPU architectures.

INSTRUCTION

Registers and memory: tiny desk vs. giant filing cabinet

Processors need places to hold values while instructions run.



R0
current value

R1
another value

Register
very small, very fast scratch space

Memory

X = 5
PRICE = 20
RESULT = ?

Large, slower, addressable storage while the program runs

Mental model: registers are the CPU's workbench. Memory is the room full of labeled boxes.

The basic moves: load, store, calculate, compare, jump

Most programs are built from many small variations of these ideas.

LOAD
copy a value from memory into a register

STORE
copy a register value back to memory

ADD / MUL
do arithmetic or logic

COMPARE
decide how two values relate

JUMP
change which instruction comes next

```
LOAD PRICE
MUL QUANTITY
STORE TOTAL
```

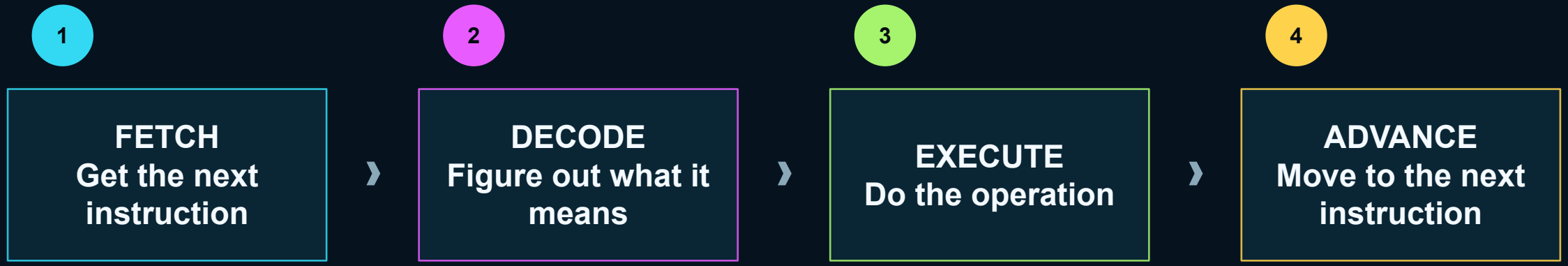
Same idea as:

total = price * quantity

```
COMPARE TOTAL, 100
JUMP_IF_GREATER discount
```

Fetch → decode → execute

A simplified cycle the processor repeats over and over.

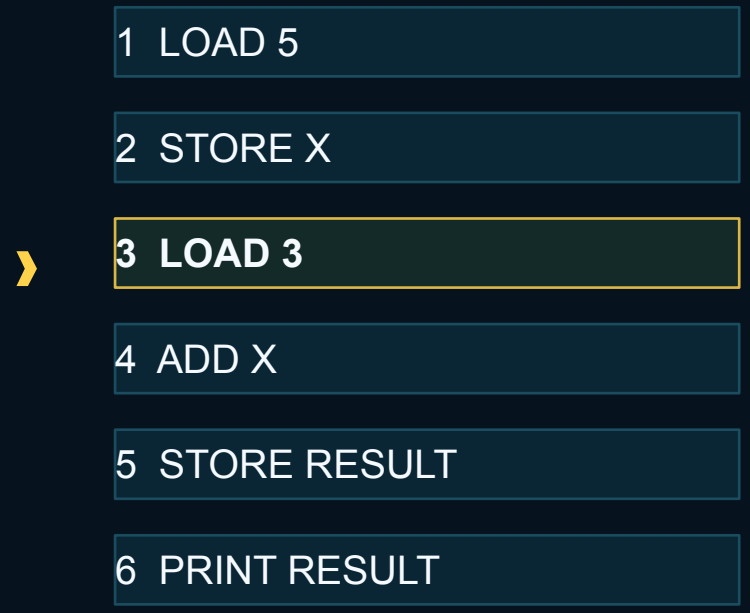


The cycle is simple. The speed and scale are the mind-blowing part.

The program counter is the bookmark

The CPU needs to know which instruction comes next.

Program Counter
“next instruction is #3”



JUMP changes the bookmark.

That is how loops and branches can happen.

Without jumps, a program would mostly be a straight line from top to bottom.

INSTRUCTION

Clock cycles, speed, and cores

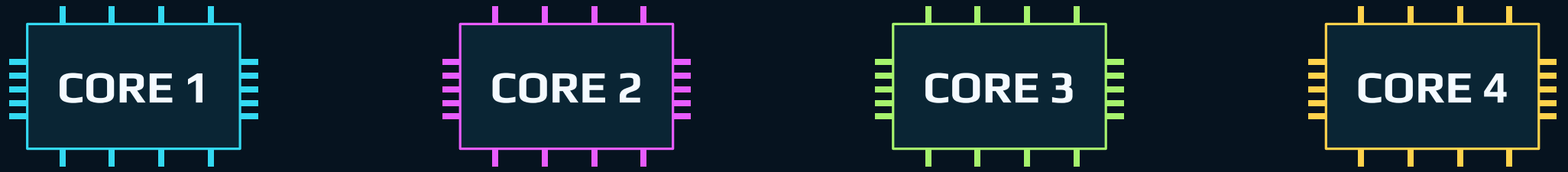
Speed is not magic; it is many tiny steps coordinated by timing.

Clock cycle
A tiny timing beat for CPU work

GHz
Billions of cycles per second

Cores
Multiple instruction engines on one chip

Important: “more GHz” does not automatically mean every program is faster. Workload, architecture, memory, and parallelism matter.



Source code vs. machine code

The same program exists at different levels of human readability.

```
total = price * quantity
```

›

```
LOAD PRICE
MUL QUANTITY
STORE TOTAL
```

›

```
01001010 00101101
11100001 00010010
00100101 10111000
```

Source code
Written for humans first

Low-level operations
Closer to hardware

Machine code
Actual CPU-friendly encoding

The processor ultimately executes machine instructions, not pretty source lines.

Compiled, interpreted, runtime: three useful words

Different languages take different routes to the processor.

Compiled language

Source code is translated ahead of time into a lower-level executable form.

Often fast to run; compile step can catch errors early.

Interpreted language

A runtime reads and executes code through another program.

Flexible and friendly; usually more layers at run time.

Virtual machine / runtime

A software environment that manages execution details for the program.

Memory, types, libraries, errors, and more.

Real systems can blur these categories. The model is still useful: code must eventually become CPU instructions.

Python’s stack of helpers

Python gives us a friendly language by adding layers above the CPU.

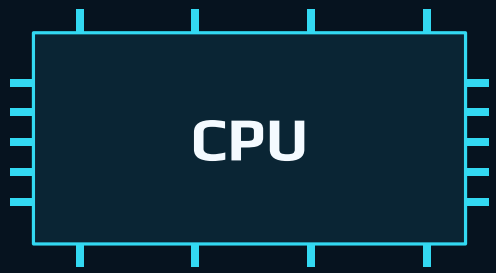
You write → Python source code

Python reads → syntax + objects + libraries

Python creates → bytecode / lower-level steps

Runtime drives → machine instructions on the CPU

Result: you can think in data, functions, loops, and names instead of registers and jumps.



A quick peek at Python bytecode

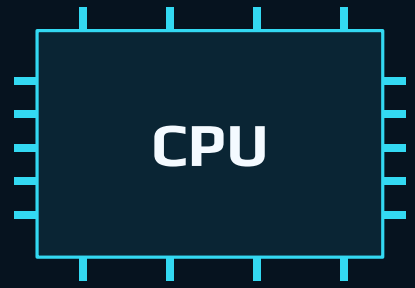
Python source is not executed as raw text forever.

```
def add(a, b):
    return a + b
```

›

```
LOAD_FAST a
LOAD_FAST b
BINARY_OP +
RETURN_VALUE
```

›



Source code
what you type

Bytecode
Python's lower-level instructions

Machine code
what the CPU runs

Bytecode is still not native CPU machine code. It is an intermediate form for the Python runtime.

The line Python lets us write

One readable line hides many tiny operations.

```
total = price * quantity
```

1. Find the object named price
2. Find the object named quantity
3. Apply multiplication rules
4. Bind the result to total
5. Eventually drive CPU instructions

Important distinction

The processor does not directly execute the characters:

“total = price * quantity”

The Python runtime performs the work needed to run it.

Why high-level languages exist

Because humans are not great at writing huge piles of tiny instructions.

Low-level code
closer to hardware
more control, more detail

Python code
closer to human intent
fewer details to manage

Runtime
does work for us
translation, memory, objects

Tradeoff: low-level code can be faster and more precise, but it is usually harder to write safely.

Python optimizes for programmer speed: readable code, rich libraries, quick iteration.

In this course, Python is the tool; today we are learning what it shields us from.

GUIDED EXERCISE

Guided exercise: become the processor

We will run a program by hand using one register and a few memory boxes.

01
Instruction list

02
Register value

03
Memory boxes

04
Output

```
LOAD 5
STORE X
LOAD 3
ADD X
STORE RESULT
PRINT RESULT
```

One register

starts empty
changes every step

Memory

X = ?
RESULT = ?

DISCUSSION

Discussion: tradeoffs

Use the paper processor experience to compare levels of code.

Which version was easier for a human to read?

Which version was closer to what a processor understands?

What work did Python perform on behalf of the programmer?

Why might low-level code run faster but be harder to develop?

Good answer pattern: “Python trades some control for readability, safety, and speed of development.”

RECAP

The five ideas to leave with

Everything in this block builds toward Python as a helpful layer above hardware.

Processor — executes small machine instructions

Registers — hold tiny working values

Memory — stores values the program can load and store

Execution cycle — fetch → decode → execute → advance

Python runtime — turns high-level source into lower-level work

EXIT TICKET: Explain why a CPU does not directly understand a line of Python source code.